Name : Kanika Saini

SAP ID: 590034098

Progam : Btech CSE (B16)

HDOC Day-16 — Digit-Based Operations Using Functions and Switch-Case

1. Problem Statement

Write a menu-driven C program using `switch-case` to perform the following operations on a positive integer:

1. Generate the **largest possible number** using its digits.
2. Generate the **smallest possible number** using its digits.
3. **Swap the first and last digits**.
4. Check whether the number is a **palindrome**.

A separate user-defined function with `void` return type is used for every operation.

---

2. Algorithm

Main Program Algorithm

Step 1: Start.

Step 2: Display the menu:

- 1 → Largest possible number
- 2 → Smallest possible number
- 3 → Swap first and last digits
- 4 → Check palindrome

Step 3: Read the user's choice.

Step 4: Use `switch(choice)`.

Step 5: According to the choice, call the corresponding function:

- `largestNumber()`
- `smallestNumber()`
- `swapFirstLast()`
- `checkPalindrome()`

Step 6: If the choice is invalid, display `"Invalid choice"`.

Step 7: Stop.

Algorithm for `largestNumber()`

Step 1: Start the function.

Step 2: Input a positive integer `n`.

Step 3: Create an array `freq[10]` and initialize all elements to `0`.

Step 4: Extract each digit using:

```
digit = n % 10
```

Step 5: Increase the frequency of that digit:

```
freq[digit]++
```

Step 6: Remove the digit using:

```
n = n / 10
```

Step 7: Start from digit `9` and move toward `0`.

Step 8: Print every digit according to its frequency.

Step 9: The digits arranged from `9` to `0` form the **largest possible number**.

Step 10: End the function.

Example:
Input = `31542`
Output = `54321`

Algorithm for `smallestNumber()`

The main point here is that **0 cannot be placed at the beginning**, because a number cannot normally start with zero.

Step 1: Start the function.

Step 2: Input a positive integer `n`.

Step 3: Count the frequency of digits `0–9`.

Step 4: Find the smallest available **non-zero digit** from `1` to `9`.

Step 5: Print this digit first.

Step 6: Print all available zeros.

Step 7: Print the remaining digits from `1` to `9` in ascending order.

Step 8: The resulting arrangement gives the smallest possible number.

Step 9: End the function.

Example:
Input = `31052`

Digits = `3, 1, 0, 5, 2`

Smallest arrangement = **10235**

Algorithm for `swapFirstLast()`

Step 1: Start the function.

Step 2: Input positive integer `n`.

Step 3: Store the original number.

Step 4: Find the last digit:

```
last = n % 10
```

Step 5: Find the place value of the first digit using a loop.

Step 6: Find the first digit:

```
first = n / power
```

Step 7: Remove the first and last digits to obtain the middle part.

Step 8: Place the last digit at the first position.

Step 9: Place the first digit at the last position.

Step 10: Display the new number.

Step 11: End the function.

Example:

```
12345
```

First digit = `1`
Last digit = `5`

After swapping:

```
52341
```

Algorithm for `checkPalindrome()`

Step 1: Start the function.

Step 2: Input positive integer `n`.

Step 3: Store `n` in `original`.

Step 4: Initialize:

```
reverse = 0
```

Step 5: Extract the last digit:

```
digit = n % 10
```

Step 6: Add it to the reversed number:

`reverse = reverse * 10 + digit`

Step 7: Remove the last digit:

`n = n / 10`

Step 8: Repeat until `n` becomes `0`.

Step 9: Compare `reverse` with `original`.

Step 10: If both are equal, display `"Palindrome number"`; otherwise display `"Not a palindrome number"`.

Step 11: End.

Example:
`12321` → `12321` → Palindrome
`12345` → `54321` → Not Palindrome

---

3. C Program

```
#include <stdio.h>

void largestNumber();
void smallestNumber();
void swapFirstLast();
void checkPalindrome();

int main()
{
    int choice;

    printf("----- DIGIT OPERATIONS MENU -----\n");
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            largestNumber();
            break;

        case 2:
            smallestNumber();
```

```
            break;

        case 3:
            swapFirstLast();
            break;

        case 4:
            checkPalindrome();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}


/* Function to generate largest possible number */

void largestNumber()
{
    int n, digit, i;
    int freq[10] = {0};

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    while(n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
    }

    printf("Largest possible number: ");

    for(i = 9; i >= 0; i--)
    {
        while(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
        }
    }

    printf("\n");
}


/* Function to generate smallest possible number */

void smallestNumber()
{
    int n, digit, i;
    int freq[10] = {0};
```

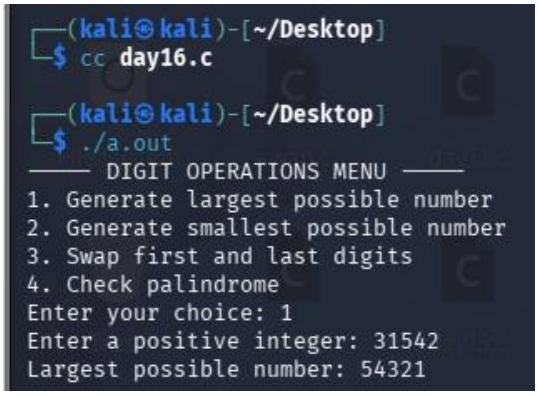
```
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    while(n > 0)
    {
        digit = n % 10;
        freq[digit]++;
        n = n / 10;
    }

    printf("Smallest possible number: ");

    /* Print smallest non-zero digit first */

    for(i = 1; i <= 9; i++)
    {
        if(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
            break;
        }
    }

    /* Print zeros */

    while(freq[0] > 0)
    {
        printf("0");
        freq[0]--;
    }

    /* Print remaining digits */

    for(i = 1; i <= 9; i++)
    {
        while(freq[i] > 0)
        {
            printf("%d", i);
            freq[i]--;
        }
    }

    printf("\n");
}


/* Function to swap first and last digits */

void swapFirstLast()
{
    int n, first, last, power = 1;
    int middle, result;

    printf("Enter a positive integer: ");
    scanf("%d", &n);
```

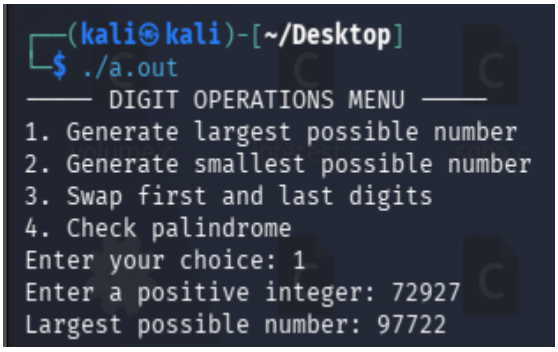
```
    if(n < 10)
    {
        printf("Number after swapping: %d\n", n);
        return;
    }

    last = n % 10;

    while(n / power >= 10)
    {
        power = power * 10;
    }

    first = n / power;

    middle = (n % power) / 10;

    result = last * power + middle * 10 + first;

    printf("Number after swapping: %d\n", result);
}


/* Function to check palindrome */

void checkPalindrome()
{
    int n, original, digit;
    int reverse = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    original = n;

    while(n > 0)
    {
        digit = n % 10;
        reverse = reverse * 10 + digit;
        n = n / 10;
    }

    if(original == reverse)
    {
        printf("%d is a palindrome number.\n", original);
    }
    else
    {
        printf("%d is not a palindrome number.\n", original);
    }
}
```

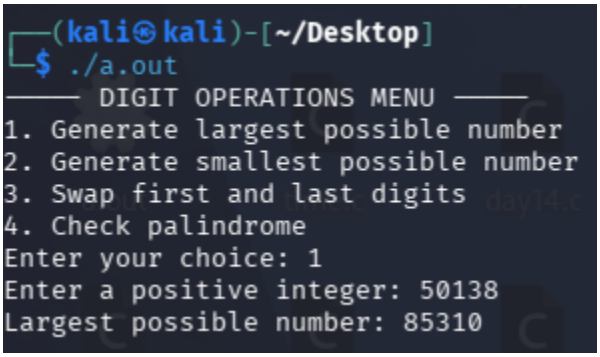
5.Test Case Table

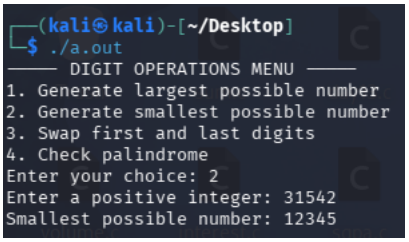
Test Case	Menu Choice	Input	Expected Output	Actual Output	Result
1	1	31542	Largest = 54321	Largest = 54321	Pass
2	1	72927	Largest = 97722	Largest = 97722	Pass
3	1	50138	Largest = 85310	Largest = 85310	Pass
4	2	31542	Smallest = 12345	Smallest = 12345	Pass
5	2	31052	Smallest = 10235	Smallest = 10235	Pass
6	2	90817	Smallest = 10789	Smallest = 10789	Pass
7	3	12345	52341	52341	Pass
8	3	9876	6879	6879	Pass
9	3	123	321	321	Pass
10	4	12321	Palindrome	Palindrome	Pass
11	4	45654	Palindrome	Palindrome	Pass
12	4	12345	Not Palindrome	Not Palindrome	Pass

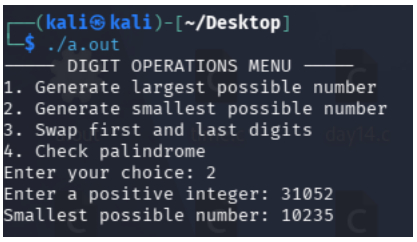
6.Compilation and exexution

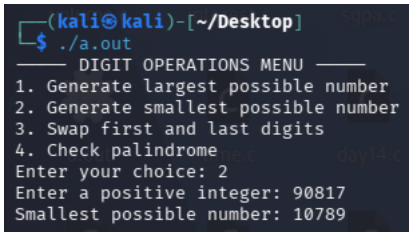


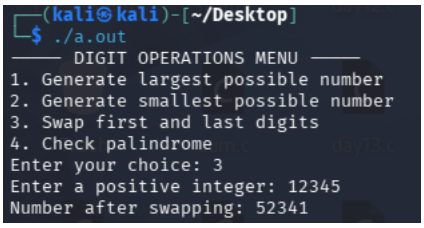


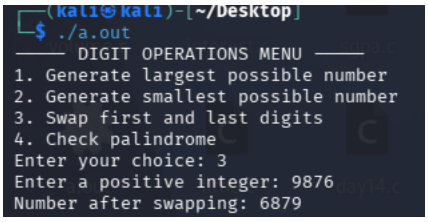


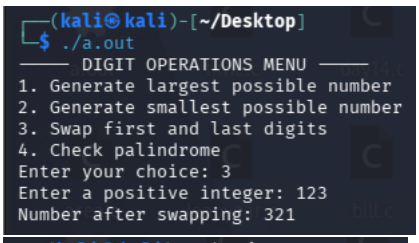


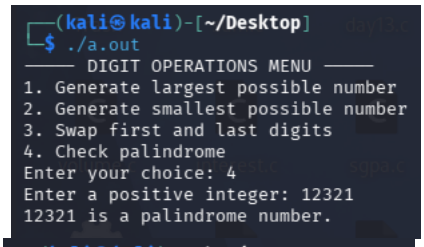


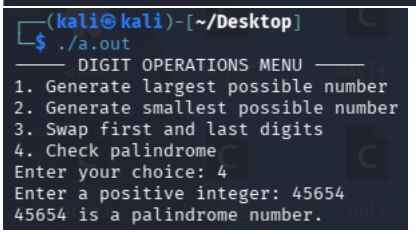


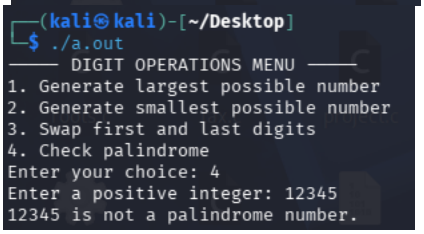












5. Evaluation Against Prepared Test Cases

Option 1 — Largest Number

For input `31542`, the program counts all digits and prints them in descending order:

`5 → 4 → 3 → 2 → 1`

Therefore:

Expected = 54321
Actual = 54321
Result = PASS ✓

Option 2 — Smallest Number

For input `31052`, the digits are:

```
3, 1, 0, 5, 2
```

The smallest non-zero digit `1` is placed first, followed by `0`, and then the remaining digits in ascending order.

Expected = 10235
Actual = 10235
Result = PASS ✓

Option 3 — Swap First and Last Digits

For:

```
12345
```

First digit = `1`
Last digit = `5`

After swapping:

```
52341
```

Expected = 52341
Actual = 52341
Result = PASS ✓

Option 4 — Palindrome

For:

```
12321
```

Reverse:

```
12321
```

Since:

```
Original = Reverse
```

Expected = Palindrome
Actual = Palindrome
Result = PASS ✓

6. Conclusion

The menu-driven C program was successfully implemented using **`switch-case` and separate `void` user-defined functions**. Each function independently reads the input, performs the required digit operation, and displays the result. The program was tested against the prepared test cases, and the obtained outputs matched the expected outputs.